

**FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO –
FECAP
GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO**

Autor(es):

Arthur Rodrigues Ferreira – 24026567

Bruno Da Silva Ribeiro – 24025958

Gabriel Henrique Coelho Marussi – 24026609

Kauan Rocha Dias – 24026492

Docente(es):

Rodinil Da Silva Moreira Lisboa

**RELATÓRIO DE ENTREGA 2:
Transformações Lineares sobre Coordenadas de Pixels**

Introdução

No Entregável 1, demonstramos que uma imagem digital é uma estrutura matricial: cada pixel é representado por um vetor RGB armazenado em uma posição (linha, coluna) da matriz. A imagem do saco de arroz Camil 1 kg (300×300 pixels) foi convertida em um CSV de 90.000 linhas, provando a equivalência entre dado visual e dado matricial.

Já no relatório entregável 2 avança sobre essa base. O objeto de estudo agora não são os valores armazenados na matriz (intensidades R, G, B), mas sim as coordenadas (x, y) de cada pixel o domínio espacial da imagem. Aplicamos Transformações Lineares $T: \mathbb{R}^2 \rightarrow \mathbb{R}^2$ que reposicionam geometricamente os pixels, alterando forma, orientação e escala da figura sem modificar suas cores.

Objetivo central: mostrar que toda transformação geométrica de uma imagem é uma multiplicação de matriz 2×2 atuando sobre o vetor de posição de cada pixel.

O Papel do Determinante

O determinante $\det(A) = ad - bc$ é o fator de escala de área da transformação:

$\det(A)$	Significado geométrico	Invertível?
> 1	Área aumentada	Sim
$= 1$	Área preservada (isometria de área)	Sim
entre 0 e 1	Área reduzida	Sim
< 0	Reflexão + escala de área	Sim
$= 0$	Colapso — dimensão inferior	Não

Mapeamento Direto e a Solução

A abordagem se traduz em iterar sobre cada pixel da imagem de entrada, calcular sua nova posição e copiá-lo para a saída. Isso é feito le produz buracos na imagem de saída, pois coordenadas de destino nem sempre são cobertas uniformemente no espaço discreto de pixels. A solução adotada é o **Mapeamento Inverso (Inverse Mapping)**: para cada pixel (x', y') da imagem de saída, calcula-se a posição de origem aplicando a matriz inversa: $[x, y]^T = A^{-1} \cdot [x', y']^T$. Isso garante que todo pixel de saída seja preenchido. A existência de A^{-1} exige $\det(A) \neq 0$ razão pela qual o Colapso Dimensional recebe tratamento especial com Mapeamento Direto.

Estrutura do Código

O script `transformacoes_lineares.py` está organizado em quatro blocos:

- **Bloco 1 — Definição das Matrizes:** funções que retornam a matriz A para cada transformação, imprimindo a matriz e seu determinante.
- **Bloco 2 — Motor de Transformação:** função `aplicar_transformacao()` implementando o mapeamento inverso vetorizado com NumPy meshgrid.
- **Bloco 3 — Visualização:** função `plotar_comparacao()` que gera as figuras com Matplotlib.
- **Bloco 4 — Execução Principal:** `main()` que orquestra todas as transformações e salva os resultados.

Transformações Aplicadas

- 1) Escalonamento
- 2) Cisalhamento
- 3) Rotação ($\theta = 45^\circ$)
- 4) Colapso Dimensional – $\text{Det}(A) = 0$

Resultados Visuais

Figura 1



Pontuação: Imagem original (saco de arroz, 300×300 px) comparada com as quatro transformações lineares.

Figura 2



Pontuação: Comparação direta entre a imagem original e a projeção ortogonal

Conclusão

A conexão entre Álgebra Linear e Visão Computacional estabelecida nestes dois entregáveis (imagens como matrizes e transformações como operadores lineares) é exatamente a base matemática de redes neurais convolucionais CNNs, renderização 3D, compressão de imagem e reconhecimento de padrões.

- Imagens como espaços vetoriais: o conjunto de coordenadas (x, y) dos pixels forma um espaço vetorial $\mathbb{R}^2$, e toda transformação geométrica é um operador linear nesse espaço.
- A matriz codifica as colunas e revelam o destino da base original.
- Determinante como diagnóstico: $\det(A)$ concentra as propriedades estruturais mais relevantes (fator de escala de área, orientação e a invertibilidade)